

DTO 처리 흐름

프론트 요청 - TMAP - GIS/알고리즘 - 추천 응답 - 상세 조회

작성 기준 | 2026-08-26

RunStop 코스 추천 과정에서 주고받는 DTO와 처리 흐름을 정리한 문서입니다.

목차

아래 항목을 클릭하면 해당 섹션으로 이동합니다.

1

프론트 요청

2

Backend에서 TMAP으로 경로 요청

3

TMAP 응답에서 Backend 형식으로 변환

4

GIS의 LineString로 변환

5

프론트의 POST 요청에 생성 완료 응답

6

프론트의 GET 요청 처리

1. 프론트 요청

RunStop App → Backend

> [POST /api/routes/recommend](#) 요청을 하게됩니다.

```
{
  "start": {
    "latitude": 37.543,
    "longitude": 127.044
  },
  "end": {
    "latitude": 37.543,
    "longitude": 127.044
  },
  "wayPoints": [
    {
      "name": "편의점",
      "coordinate": {
        "latitude": "float",
        "longitude": "float"
      }
    }
  ],
  // LOOP / ONE_WAY / ROUND_TRIP 중 하나
  "routeType": "LOOP",
  // 사용자가 원하는 거리
  "targetDistanceKm": 7.0,
  // 허용 거리 오차 % (아직 후보)
  "distanceTolerance": 10,
  // 필수 조건
  "requirements": {
    "toilet": true
  },
  // 가중치는 데이터 수집 후 28~31일에 명확히 설정
  "weights": {
    "distance": 5,
    "elevation": 4,
    "toilet": 5,
    "store": 2,
    "park": 3,
    "night": 5
  }
}
```

[목차로 돌아가기](#)

2. Backend에서 TMAP으로 경로 요청

Backend → TMAP

> 프론트에서 받은 출발지/목적지와 경유지를 TMAP API 규격으로 변환하여 호출합니다.

```
{
  "start": {
    "latitude": 37.543,
    "longitude": 127.044
  },
  "end": {
    "latitude": 37.543,
    "longitude": 127.044
  },
  "waypoints": [
    {
      "latitude": 37.545,
      "longitude": 127.047
    }
  ]
}
```

사용 위치

- 입력: 프론트에서 받은 출발지 / 목적지 / 경유지 좌표
- 처리: Backend에서 TMAP API가 요구하는 형식으로 변환
- 목적: 실제 보행 가능한 경로를 TMAP에 요청

[목차로 돌아가기](#)

3. TMAP 응답에서 Backend 형식으로 변환

TMAP → Backend

> 응답을 서비스에서 사용 가능한 형태로 변환합니다.

```
{
  "routeId": 101,
  "distanceKm": 7.1,
  "coordinates": [
    {
      "latitude": 37.5430,
      "longitude": 127.0440
    },
    {
      "latitude": 37.5432,
      "longitude": 127.0445
    }
  ]
}
```

사용 위치

- routeId: 서비스 내부에서 후보 경로를 식별하는 값
- distanceKm: 생성된 경로의 거리
- coordinates[]: 이후 GIS의 LineString으로 변환할 경로 위경도 목록

[목차로 돌아가기](#)

4. GIS의 LineString로 변환

PostGIS + 알고리즘

> PostGIS를 이용하여 LineString로 변환하여 buffer 및 점수화를 진행합니다.

- PostGIS를 공부한 뒤 해결할 영역입니다.
- PostGIS에서 Point와 LineString의 거리를 계산할 수 있는 기능을 제공한다고 확인하였습니다.
- 기본 좌표계는 EPSG:4326(위경도)이며 거리 계산시에 EPSG:5179(평면 좌표)입니다.
- 이 부분에서 알고리즘과 함께 경로를 계산하여 DB에 저장하게 됩니다.
- 유사 경로는 제거하며 점수화를 통해 후보 3군 생성 또는 완벽한 결과가 없음을 나타내게 됩니다.

알고리즘 및 GIS 처리 계층

처리 단계	사용 내용
입력	Backend Route의 coordinates[]
GIS 형식	LineString / Point
좌표계	EPSG:4326 → EPSG:5179
공간 처리	buffer / Point와 LineString 거리 계산
알고리즘	점수화 / 유사 경로 제거 / 후보 3군 생성 또는 완벽한 결과 없음
저장	경로 계산 결과를 DB에 저장

[목차로 돌아가기](#)

5. 프론트의 POST 요청에 생성 완료 응답

Backend → RunStop App

> 결과물을 즉시 한번에 앵고 생성 결과를 반환합니다. (POST /api/routes/recommend) routeId는 데이터베이스의 route_recommandation.idx와 동일합니다. (2026-08-26 기준)

```
{
  "success": true,
  "data": {
    "exactMatch": true,
    "routes": [
      {
        "routeId": 101,
        "distanceKm": 7.1,
        "conditionScore": 89.7
      }
    ]
  },
  "message": "추천 코스 조회가 완료되었습니다."
}
```

응답에서 확인하는 값

- exactMatch: 생성 결과가 완벽한 결과인지 나타내는 값
- routes[]: 생성된 후보 경로 목록
- routeId: 이후 GET 상세 조회에 사용하는 경로 식별자
- distanceKm / conditionScore: 추천 목록에서 확인할 경로 요약값

[목차로 돌아가기](#)

6. 프론트의 GET 요청 처리

RunStop App ↔ Backend

> POST 응답을 바탕으로 프론트가 보낸 `GET /api/routes/{routeId}` 요청을 처리합니다.

```
{
  "success": true,
  "data": {
    "routeId": 101,
    "routeName": "서울숲 순환 코스 A",
    "distanceKm": 7.1,
    "estimatedMinutes": 48,
    "elevationGain": 34,
    "conditionScore": 89.7,

    // 적용된 시설들 point
    "facilities": {
      "toilets": [
        {
          "name": "서울숲 공중화장실",
          "distanceFromRoute": 32,
          "latitude": 37.543,
          "longitude": 127.044
        }
      ],
      "stores": [
        {
          "name": "CU 서울숲점",
          "distanceFromRoute": 45,
          "latitude": 37.544,
          "longitude": 127.045
        }
      ]
    },

    // 경로 위경도 목록
    "routeCoordinates": [
      {
        "latitude": 37.543,
        "longitude": 127.044
      }
    ],

    // 아마 요소별 점수를 반환하는 형태로 변환될 것 같습니다.
    "recommendReasons": [
      "목표 거리와 유사합니다.",
      "누적 상승고도가 낮습니다.",
      "야간 조명 인프라가 풍부합니다."
    ]
  }
}
```

[목차로 돌아가기](#)